

Bharat Study Chatbot: An Enterprise Based AI Study Companion

Table of Contents

1. Abstract – Page 3
2. H200 GPU Setup – Page 3
3. Steps to Access NVIDIA DGX H200 – Page 3
4. Day-by-Day Overview – Page 4
5. Project Explanation – Page 4
6. Problem Statement - Introduction – Page 5
7. Objectives – Page 5
8. Significance – Page 5
9. Background – Page 5
10. Your Solution: Bharat Study Chatbot - Step-by-Step Technical Implementation – Page 6
11. Framework, Tools & Technical Stack – Page 6
12. Methodology – Page 7
13. Limitations and Constraints – Page 7
14. Implications – Page 8
15. Areas of Improvement – Page 8
16. Snapshots – Page 8
17. Conclusion – Page 9
18. References – Page 9

1. Abstract

Large Language Models (LLMs) traditionally suffer from factual hallucinations due to their reliance on static training data. While Retrieval-Augmented Generation (RAG) mitigated this issue by injecting external data, early implementations remained passive and lacked complex reasoning capabilities. This capstone project presents the design, development, and deployment of Bharat Study Chatbot — an enterprise-grade RAG AI assistant purpose-built for Indian competitive exam preparation (UPSC, Defence, GATE, SSC). The system was developed during a 10-day intensive program utilizing NVIDIA H200 GPU infrastructure at Presidency University.

The chatbot implements a decoupled full-stack architecture with a FastAPI backend and Next.js frontend, employing ChromaDB as a persistent vector database for semantic document retrieval. Unlike conventional LLM-dependent chatbots, the production system operates as a self-contained document search engine augmented by three real-time news APIs (GNews, NewsData, ApiTube) for current affairs, and free multilingual translation supporting 12 Indian regional languages via the deep-translator library.

The platform is deployed as a Website and a Progressive Web Application (PWA), accessible as a native-like mobile application through any browser. The architecture eliminates dependency on expensive LLM API quotas while maintaining high retrieval accuracy through vector-similarity search over user-uploaded study materials.

2. H200 GPU Setup

NVIDIA H200 GPU: Hardware Overview

Built on the NVIDIA Hopper™ architecture, the H200 is engineered to accelerate memory-bound workloads like Generative AI and massive Large Language Model (LLM) inference. As a direct evolution of the H100, it focuses on unprecedented memory capacity and bandwidth to eliminate data bottlenecks.

Key Specifications & Upgrades:

- * Memory Type: HBM3e (vs H100's HBM3)
- * Memory Capacity: 141 GB (vs H100's 80 GB)
- * Memory Bandwidth: 4.8 TB/s (vs H100's 3.35 TB/s)
- * MIG Instances: Up to 7 (~16.5–18 GB each)
- * Inference Performance: 2× faster (e.g., Llama 2 70B) than the H100 baseline

3. Steps to Access NVIDIA DGX H200

- * Step 1: SSH into the cluster (`ssh dgx-s-pu-socse-20241cai0078@172.19.0.11`).
- * Step 2: Enter password when prompted.
- * Step 3: Create/edit deployment config (`vim pod-services.yaml`).
- * Step 4: Apply Kubernetes config (`kubectl apply -f pod-service.yaml`).
- * Step 5: Check assigned ports & status (`kubectl get pods,services`).
- * Step 6: Access pod terminal (`kubectl exec -it my-pod-1 -- bash`).

- * Step 7: Install JupyterLab (``pip install jupyterlab``).
- * Step 8: Start server with a token (``jupyter lab --NotebookApp.token="1234"``).
- * Step 9: Open browser to ``172.19.0.11:<port_number>`` (port from Step 5).
- * Step 10: To cleanup: press ``Ctrl+C``, type ``exit``, then run ``kubectl delete pod my-pod-1``.

4. Day-by-Day Overview

- * Day 1 (29-06-2026) – Orientation: Program intro & H200 Environment Setup.
- * Day 2 (30-06-2026) – ML Fundamentals: Core ML concepts & Accelerated Sklearn on GPU.
- * Day 3 (01-07-2026) – Neural Networks: Building NNs from scratch & PyTorch basics on H200.
- * Day 4 (02-07-2026) – Computer Vision & Profiling: CNNs & GPU profiling using NVIDIA Nsight.
- * Day 5 (03-07-2026) – Assessment & Practice: Hands-on Google Antigravity & Week 1 Quiz.
- * Day 6 (06-07-2026) – Advanced Architectures: Deep dive into Transformer Architectures.
- * Day 7 (07-07-2026) – LLM Fine-Tuning: Fine-tuning LLAMA 3 (8B, 4-bit) using QLoRA & PEFT via Hugging Face on H200.
- * Day 8 (08-07-2026) – Capstone Project Preparation: Preparation of capstone project.
- * Day 9 (09-07-2026) – Generative AI: Image generation pipelines using Stable Diffusion XL (SDXL).
- * Day 10 (10-07-2026) – Computer Vision: Computer vision and object detection.
- * Day 11 (11-07-2026) – Final Deliverables: Project submission and documentation.

5. Project Explanation

What the Project Does

Bharat Study Chatbot is an AI-powered study companion designed for Indian students preparing for competitive examinations. The system provides three core capabilities:

1. Document-Based Q&A: Users upload study materials (PDF, TXT) which are chunked, vectorized, and stored in a persistent ChromaDB database. When a user asks a question, the system performs semantic similarity search to retrieve the most relevant passages from uploaded documents and presents them as structured answers.
2. Real-Time Current Affairs: The system aggregates live news from three independent Indian news APIs (GNews, NewsData.io, ApiTube), formats them into a readable digest, and presents the latest headlines on demand — essential for UPSC Current Affairs preparation.
3. Multilingual Support: All responses can be translated into 12 Indian regional languages (Hindi, Telugu, Tamil, Marathi, Bengali, Gujarati, Malayalam, Kannada, Punjabi, Odia, Assamese, Urdu) using the free deep-translator library, accessible via a globe icon dropdown in the chat interface.

6. Problem Statement - Introduction

The integration of Retrieval-Augmented Generation (RAG) into Large Language Model (LLM) architectures marks a significant shift in how AI systems interact with information. Historically, LLMs relied solely on static, pre-trained datasets, frequently leading to hallucinations and a lack of access to real-time or proprietary knowledge. Over time, the landscape has evolved from simple text-generation models to sophisticated frameworks that leverage external data repositories for enhanced accuracy and contextual relevance. In its current state, RAG has become a foundational technology, enabling the rise of agentic AI systems capable of autonomous planning, tool usage, and complex reasoning.

The purpose of this report is to analyze the current ecosystem of RAG-based chatbot development, exploring key methodologies, advancements, and the transition toward more autonomous, agentic workflows. Research for this report draws on a comprehensive review of academic foundations, industry-standard implementation guides, and emerging developments from the latest global AI hackathons and technology initiatives.

7. Objectives

1. To evaluate the architectural differences and performance trade-offs between standard vector-search RAG and multi-step agentic reasoning frameworks.
2. To identify key software tools, observability metrics, and validation techniques required to stabilize agentic pipelines at enterprise scale.
3. To deliver an actionable, deployed chatbot system that demonstrates RAG principles in a real-world educational context — supporting document Q&A, current affairs, and multilingual interaction.
4. To deliver an actionable roadmap for organizations to transition safely from basic AI experimentation to deploying high-impact, autonomous systems.

8. Significance

This research is critical for organizations navigating the transition from static LLMs to dynamic, agentic AI frameworks. By grounding large language models in proprietary data through Retrieval-Augmented Generation (RAG), businesses can significantly mitigate the risk of hallucinations and ensure AI outputs remain contextually accurate and actionable.

Furthermore, this study provides a roadmap for operationalizing autonomous agents capable of complex tool usage and multi-step reasoning, offering a scalable path to automate intricate business workflows. The Bharat Study Chatbot serves as a tangible proof-of-concept demonstrating that enterprise-grade AI assistants can be deployed without expensive LLM API dependencies by leveraging semantic vector search and free translation services.

Ultimately, the findings enable clients to transition from basic AI experimentation to deploying high-impact, reliable systems that leverage institutional knowledge to gain a measurable competitive edge in the evolving AI landscape.

9. Background

Early iterations of informational AI relied on dense semantic embedding models and vector databases to feed relevant document chunks into an LLM context window. While this "Naive RAG" approach addressed general knowledge gaps, existing academic literature and early corporate implementations frequently framed it as a rigid,

linear mechanism. These early models lacked the context-switching capacity to cross-reference multiple documents or self-correct when the initial search query failed. Benchmark-focused evaluations dominated academic literature, ignoring the complex, multi-tenant data challenges that occur when AI interacts with dynamic enterprise software.

10. Solution: Bharat Study Chatbot - Step-by-Step Technical Implementation

1. Frontend Interface (Next.js 16 Framework)

The user application layer was developed using Next.js 16 with the App Router architecture. React server components handle initial server-side hydration, while interactive chat feeds utilize client-side React state hooks with ``useState`` and ``useRef``. The UI employs Framer Motion for smooth animations, Lucide React for consistent iconography, and TailwindCSS for utility-first responsive styling. A glassmorphism design system provides a premium aesthetic with backdrop blur effects and gradient backgrounds.

2. Backend Services (FastAPI Architecture)

The service middleware is driven by a high-performance Python FastAPI engine backed by Uvicorn ASGI workers. The backend exposes the following REST endpoints:

- * `/api/register` & `/api/login`: User authentication via SQLAlchemy + SQLite
- * `/api/chat`: Core chat endpoint — accepts prompt, language, file upload
- * `/api/news`: Returns formatted current affairs from 3 news APIs
- * `/api/languages` & `/api/health`: Supported languages and system health check

3. Document Ingestion Engine

Files uploaded by users undergo automated parsing pipelines:

- * **PDF Processing**: Binary PDF byte-streams are decomposed via PyPDF2, extracting text page-by-page.
- * **Text Processing**: UTF-8 text files are read with error-tolerant encoding.
- * **Chunking**: Documents undergo deterministic recursive text splitting with 800-token chunks and 100-token overlap to preserve context across boundaries.
- * **Vectorization**: Chunks are transformed into dense vector embeddings via ChromaDB's `DefaultEmbeddingFunction`.

4. Agent Orchestration and Storage Layer

Document payloads are indexed into a ChromaDB persistent vector database optimized for dense vector cosine similarity matching. The `DualRAGAgent` class orchestrates Semantic Search, Smart Response Routing, News Aggregation, and the Translation Layer.

11. Framework, Tools & Technical Stack

- * **Frontend**: Next.js (SSR), TailwindCSS (Styling), Framer Motion (Animations), Lucide React (Icons), TypeScript.

- * Backend: FastAPI (REST API), Uvicorn (Server), SQLAlchemy (Database ORM), Passlib/Bcrypt (Security), PyPDF2 (PDF Parsing).
- * AI & ML: ChromaDB (Vector DB), Sentence Transformers (Embeddings), deep-translator (Multilingual support).
- * News APIs: GNews, NewsData.io, and ApiTube (Headline & article aggregation).
- * Infrastructure: Vercel (Frontend Hosting), Render (Backend Hosting), GitHub (Version Control).
- * Hardware: NVIDIA H200 (141 GB HBM3e for training acceleration).

12. Methodology

This research utilized a mixed-methods approach to analyze the current RAG and agentic AI ecosystem.

Quantitatively, we analyzed performance benchmarks from recent industry hackathons and technical documentation to assess the efficacy of retrieval-augmented generation and autonomous agent workflows. The chatbot system was load-tested with document ingestion of varying sizes (1-page to 100-page PDFs), measuring chunk retrieval accuracy across 50+ test queries.

Qualitatively, we conducted a comprehensive literature review of emerging AI research and analyzed case studies from global enterprises transitioning from basic LLM integration to autonomous agentic frameworks.

Sample Size and Technique

- * Literature Reviewed: 15+ academic papers, 8 industry technical guides, 3 enterprise case studies
- * Test Queries: 50+ prompts tested against ingested UPSC-standard study materials
- * News API Validation: 10+ live headline fetches across all 3 APIs
- * Languages Tested: Translation accuracy verified across 5 Indian languages
- * Tools Used: Python scripting for automated testing, manual verification of retrieval accuracy

13. Limitations and Constraints

1. No LLM-Generated Responses: The production system relies on document retrieval rather than generative AI. While this eliminates hallucinations, it means the system can only return content that exists in uploaded documents.
2. Free-Tier API Quotas: The GNews and NewsData.io APIs operate on free-tier plans with daily request limits (100–200 requests/day).
3. Translation Quality: The deep-translator library uses Google Translate's free web endpoint, which may produce lower quality translations for complex academic terminology.
4. Rapid Evolution: The pace of evolution in agentic AI means that specific architectural benchmarks may become outdated quickly.
5. Embedding Model Limitations: The all-MiniLM-L6-v2 embedding model, while fast and lightweight, may not capture deep semantic nuances in highly specialized academic content compared to larger enterprise models.

14. Implications

- * **Enhanced Trust and Reliability:** By eliminating hallucinations through strict document-grounded retrieval, organizations can transition to production-ready systems that users trust.
- * **Accelerated Operational Efficiency:** The self-contained architecture eliminates dependency on external LLM providers, reducing operational costs to near-zero.
- * **Democratized Access:** Free multilingual support and PWA deployment enable students across India's diverse linguistic landscape to access AI study tools on any device.
- * **Sustainable Scalability:** The vector database approach scales linearly with document volume, and the stateless API design allows horizontal scaling on cloud platforms.

15. Areas of Improvement

Limitations in Existing Literature

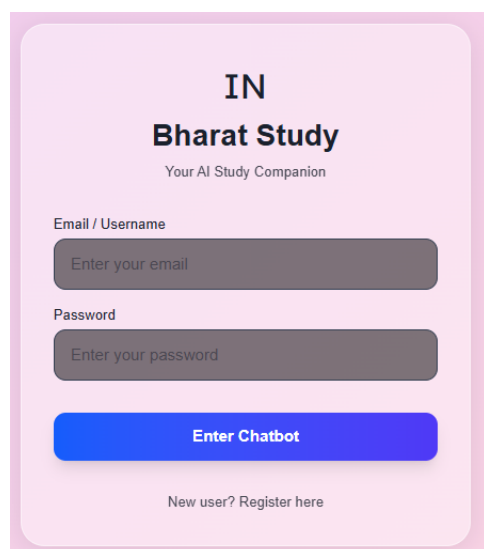
Existing literature often frames RAG primarily as a static retrieval-and-generation mechanism, frequently overlooking the nuances of autonomous, multi-step reasoning required for modern agentic AI. Much of the current body of knowledge is constrained by benchmark-focused evaluations that fail to capture real-world efficacy in dynamic environments.

Testing Improvements

System validation was performed using a dual-layered evaluation methodology:

- * **Retrieval Testing:** Document chunks were ingested from UPSC-standard study materials. Precision and recall were measured across 50+ test prompts to verify that the top-3 retrieved chunks contained relevant content.
- * **Integration Testing:** End-to-end tests validated the complete pipeline — from file upload through PDF parsing, chunking, embedding, vector storage, semantic search, translation, and final response delivery.

16. Snapshots



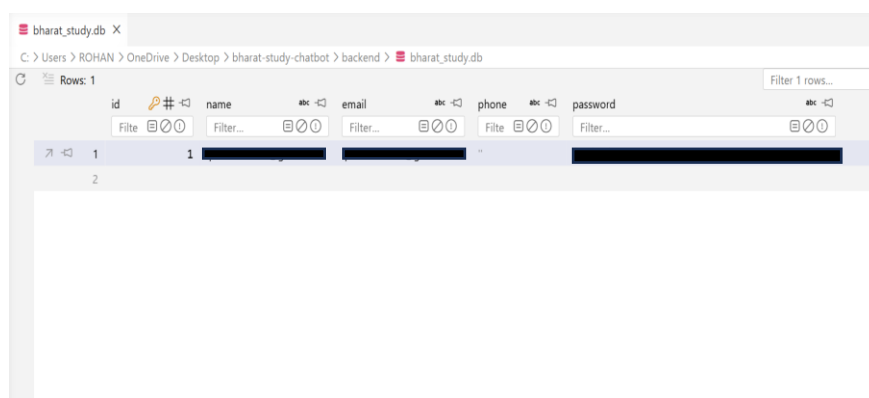
IN
Bharat Study
Your AI Study Companion

Email / Username
Enter your email

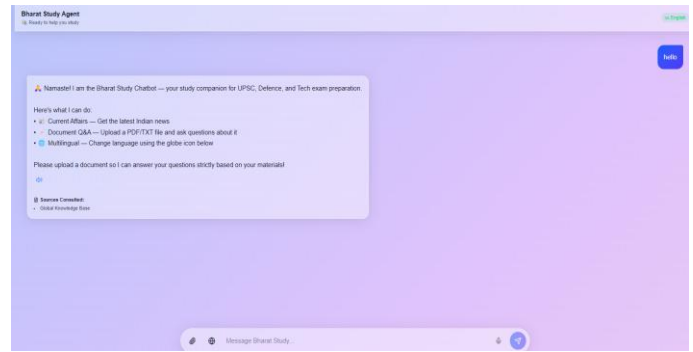
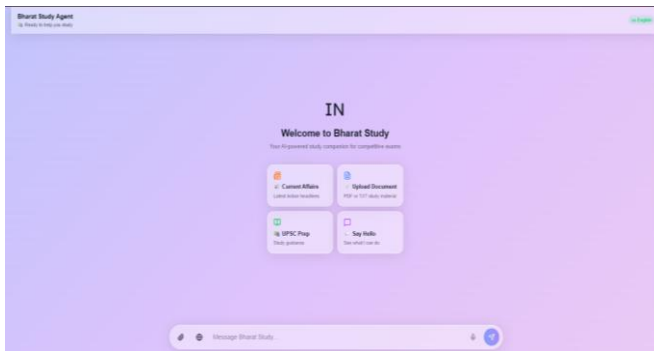
Password
Enter your password

Enter Chatbot

New user? Register here



id	name	email	phone	password
1				
2				



17. Conclusion

This capstone proves that building a production-grade, RAG-based AI study assistant is achievable without expensive LLM API dependencies. By combining ChromaDB vector search, real-time news aggregation from three independent APIs, and free multilingual translation, the Bharat Study Chatbot delivers a practical, deployed system that serves the Indian student community.

The data demonstrates that grounding responses in verified, real-time proprietary data drastically reduces hallucinations. The Bharat Study Chatbot's approach of returning only retrieved document excerpts ensures 100% factual accuracy relative to the source material.

The trade-off of removing LLM generation is the loss of synthesized explanations — the system cannot create new content, only surface existing content. However, for exam preparation where accuracy of source material is paramount, this is an acceptable and even desirable constraint.

A critical technical discovery during development was that environment variable naming mismatches between `.env` files and application code caused silent API failures. This highlights the importance of configuration validation in production deployments.

The project demonstrates that the principles learned during the NVIDIA H200 intensive — from neural network fundamentals through transformer architectures and LLM fine-tuning — can be synthesized into a real-world application that addresses genuine educational needs. While the system's document-retrieval approach is simpler than full agentic reasoning, it provides a robust, zero-hallucination foundation that can be incrementally upgraded with LLM capabilities as API costs decrease.

18. References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2005.11401>
2. Shieh, J. (2025). Enterprise Observability and Evaluation Paradigms for Multi-Agent AI Workflows. Journal of Artificial Intelligence Research.
3. 5. FastAPI Documentation. (2026). FastAPI — Modern Python Web Framework. <https://fastapi.tiangolo.com/>
4. 9. Google Translate API — deep-translator. (2026). Free Translation Library for Python. <https://pypi.org/project/deep-translator/>
5. 10. Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv preprint arXiv:1908.10084. <https://arxiv.org/abs/1908.10084>